

```
In [2]: import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from datasets import load_dataset, concatenate_datasets, DatasetDict
from transformers import BertTokenizer
from tqdm.auto import tqdm
import numpy as np
import torch.nn.functional as F
import warnings
warnings.filterwarnings('ignore')
```

```
In [3]: device = torch.device("cuda:3" if torch.cuda.is_available() else "cpu")
device
print(device)

SEED = 1234
torch.manual_seed(SEED)
torch.backends.cudnn.deterministic = True

torch.cuda.get_device_name(0)
```

cuda:3

```
Out[3]: 'NVIDIA GeForce RTX 2080 Ti'
```

Copy Previous class architecture details from task 1

```
In [4]: class Embedding(nn.Module):
    def __init__(self, vocab_size, max_len, n_segments, d_model, device):
        super(Embedding, self).__init__()
        self.tok_embed = nn.Embedding(vocab_size, d_model) # token embedding
        self.pos_embed = nn.Embedding(max_len, d_model) # position embedding
        self.seg_embed = nn.Embedding(n_segments, d_model) # segment(token) embedding
        self.norm = nn.LayerNorm(d_model)
        self.device = device

    def forward(self, x, seg):
        #x, seg: (bs, len)
        seq_len = x.size(1)
        pos = torch.arange(seq_len, dtype=torch.long).to(self.device)
        pos = pos.unsqueeze(0).expand_as(x) # (len,) -> (bs, len)
        embedding = self.tok_embed(x) + self.pos_embed(pos) + self.seg_embed(seg)
        return self.norm(embedding)
```

```
In [5]: def get_attn_pad_mask(seq_q, seq_k, device):
    batch_size, len_q = seq_q.size()
    batch_size, len_k = seq_k.size()
    # eq(zero) is PAD token
    pad_attn_mask = seq_k.data.eq(0).unsqueeze(1).to(device) # batch_size x 1 x len_k
    return pad_attn_mask.expand(batch_size, len_q, len_k) # batch_size x len_q x len_k
```

```
In [6]: class EncoderLayer(nn.Module):
    def __init__(self, n_heads, d_model, d_ff, d_k, device):
        super(EncoderLayer, self).__init__()
        self.enc_self_attn = MultiHeadAttention(n_heads, d_model, d_k, device)
        self.pos_ffn = PoswiseFeedForwardNet(d_model, d_ff)

    def forward(self, enc_inputs, enc_self_attn_mask):
        enc_outputs, attn = self.enc_self_attn(enc_inputs, enc_inputs, enc_self_attn_mask)
        enc_outputs = self.pos_ffn(enc_outputs) # enc_outputs: [batch_size x d_model]
        return enc_outputs, attn
```

```
In [7]: class ScaledDotProductAttention(nn.Module):
    def __init__(self, d_k, device):
        super(ScaledDotProductAttention, self).__init__()
        self.scale = torch.sqrt(torch.FloatTensor([d_k])).to(device)

    def forward(self, Q, K, V, attn_mask):
        scores = torch.matmul(Q, K.transpose(-1, -2)) / self.scale # scores: [batch_size x d_model x d_model]
        scores.masked_fill_(attn_mask, -1e9) # Fills elements of self tensor with value -1e9 where attn_mask == 1
        attn = nn.Softmax(dim=-1)(scores)
        context = torch.matmul(attn, V)
        return context, attn
```

```
In [8]: class MultiHeadAttention(nn.Module):
    def __init__(self, n_heads, d_model, d_k, device):
        super(MultiHeadAttention, self).__init__()
        self.n_heads = n_heads
        self.d_model = d_model
        self.d_k = d_k
        self.d_v = d_k
        self.W_Q = nn.Linear(d_model, d_k * n_heads)
        self.W_K = nn.Linear(d_model, d_k * n_heads)
        self.W_V = nn.Linear(d_model, self.d_v * n_heads)

        # FIX: Renamed to W_0 to match your Task 1 weights and fix AttributeError: 'MultiHeadAttention' object has no attribute 'W_0'
        self.W_0 = nn.Linear(n_heads * d_k, d_model)

        self.scaled_dot_attn = ScaledDotProductAttention(d_k, device)
        self.layer_norm = nn.LayerNorm(d_model)
        self.device = device

    def forward(self, Q, K, V, attn_mask):
        residual, batch_size = Q, Q.size(0)
        q_s = self.W_Q(Q).view(batch_size, -1, self.n_heads, self.d_k).transpose(1, 2)
        k_s = self.W_K(K).view(batch_size, -1, self.n_heads, self.d_k).transpose(1, 2)
        v_s = self.W_V(V).view(batch_size, -1, self.n_heads, self.d_v).transpose(1, 2)

        attn_mask = attn_mask.unsqueeze(1).repeat(1, self.n_heads, 1, 1)

        context, attn = self.scaled_dot_attn(q_s, k_s, v_s, attn_mask)
        context = context.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)

        # FIX: Ensure it uses the correct attribute name
        context = self.W_0(context)
```

```

        output = self.W_0(context)
        return self.layer_norm(output + residual), attn

```

```

In [9]: class PoswiseFeedForwardNet(nn.Module):
        def __init__(self, d_model, d_ff):
            super(PoswiseFeedForwardNet, self).__init__()
            self.fc1 = nn.Linear(d_model, d_ff)
            self.fc2 = nn.Linear(d_ff, d_model)

        def forward(self, x):
            # (batch_size, len_seq, d_model) -> (batch_size, len_seq, d_ff) -> (batch_size, len_seq, d_model)
            return self.fc2(F.gelu(self.fc1(x)))

```

```

In [10]: class BERT(nn.Module):
        def __init__(self, n_layers, n_heads, d_model, d_ff, d_k, n_segments, vocab_size, max_len):
            super(BERT, self).__init__()
            self.params = {'n_layers': n_layers, 'n_heads': n_heads, 'd_model': d_model, 'd_ff': d_ff, 'd_k': d_k, 'n_segments': n_segments, 'vocab_size': vocab_size, 'max_len': max_len}
            self.embedding = Embedding(vocab_size, max_len, n_segments, d_model)
            self.layers = nn.ModuleList([EncoderLayer(n_heads, d_model, d_ff, d_k) for _ in range(n_layers)])
            self.fc = nn.Linear(d_model, d_model)
            self.activ = nn.Tanh()
            self.linear = nn.Linear(d_model, d_model)
            self.norm = nn.LayerNorm(d_model)
            self.classifier = nn.Linear(d_model, 2)
            # decoder is shared with embedding layer
            embed_weight = self.embedding.tok_embed.weight
            n_vocab, n_dim = embed_weight.size()
            self.decoder = nn.Linear(n_dim, n_vocab, bias=False)
            self.decoder.weight = embed_weight
            self.decoder_bias = nn.Parameter(torch.zeros(n_vocab))
            self.device = device

        def forward(self, input_ids, segment_ids, masked_pos):
            output = self.embedding(input_ids, segment_ids)
            enc_self_attn_mask = get_attn_pad_mask(input_ids, input_ids, self.device)
            for layer in self.layers:
                output, enc_self_attn = layer(output, enc_self_attn_mask)
            # output : [batch_size, len, d_model], attn : [batch_size, n_heads, len, len]

            # 1. predict next sentence
            # it will be decided by first token(CLS)
            h_pooled = self.activ(self.fc(output[:, 0])) # [batch_size, d_model]
            logits_nsp = self.classifier(h_pooled) # [batch_size, 2]

            # 2. predict the masked token
            masked_pos = masked_pos[:, :, None].expand(-1, -1, output.size(-1))
            h_masked = torch.gather(output, 1, masked_pos) # masking position [batch_size, n_segments, d_model]
            h_masked = self.norm(F.gelu(self.linear(h_masked)))
            logits_lm = self.decoder(h_masked) + self.decoder_bias # [batch_size, n_segments, n_vocab]

            return logits_lm, logits_nsp, output

        def get_last_hidden_state(self, input_ids, segment_ids):

```

```

        output = self.embedding(input_ids, segment_ids)
        enc_self_attn_mask = get_attn_pad_mask(input_ids, input_ids, self.de
    for layer in self.layers:
        output, enc_self_attn = layer(output, enc_self_attn_mask)

    return output

```

```
In [11]: tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

Load and prepare data

```
In [12]: def load_and_prepare_data():
    snli = load_dataset('snli')
    mnli = load_dataset('glue', 'mnli')

    # Filter invalid labels [cite: 3, 7]
    snli = snli.filter(lambda x: x['label'] != -1)

    # Remove 'idx' from MNLI to match SNLI format [cite: 3, 4]
    for split in mnli.keys():
        if 'idx' in mnli[split].column_names:
            mnli[split] = mnli[split].remove_columns('idx')

    # Merge datasets and take a subset (100k samples as per Task 1 guideline
    raw_dataset = DatasetDict({
        'train': concatenate_datasets([snli['train'], mnli['train']]).shuffle(
        'validation': concatenate_datasets([snli['validation'], mnli['valida
    })
    return raw_dataset

```

```
In [13]: def preprocess_function(examples):
    max_seq_length = 128
    # Tokenize premise and hypothesis separately for Siamese structure [cite
    p = tokenizer(examples['premise'], padding='max_length', max_length=max_
    h = tokenizer(examples['hypothesis'], padding='max_length', max_length=m
    return {
        "premise_input_ids": p["input_ids"],
        "premise_attention_mask": p["attention_mask"],
        "hypothesis_input_ids": h["input_ids"],
        "hypothesis_attention_mask": h["attention_mask"],
        "labels": examples["label"]
    }

```

```
In [14]: class SiameseBERT(nn.Module):
    def __init__(self, bert_model, d_model):
        super(SiameseBERT, self).__init__()
        self.bert = bert_model
        self.classifier = nn.Linear(d_model * 3, 3)

    def mean_pool(self, sequence_output, attention_mask):
        # sequence_output: (batch, seq_len, hidden) -> (4, 128, 768)
        mask = attention_mask.unsqueeze(-1).expand(sequence_output.size()).f
        # sum along dimension 1 (the 128 sequence length)

```

```

        sum_embeddings = torch.sum(sequence_output * mask, dim=1)
        sum_mask = torch.clamp(mask.sum(dim=1), min=1e-9)
        return sum_embeddings / sum_mask

    def forward(self, p_ids, p_mask, h_ids, h_mask):
        #we pass zeros for segment_ids since we are processing sentences in
        p_seg = torch.zeros_like(p_ids).to(p_ids.device)
        h_seg = torch.zeros_like(h_ids).to(h_ids.device)

        #optimized: sse the encoder directly to avoid MLM/NSP logic
        p_out = self.bert.get_last_hidden_state(p_ids, p_seg)
        h_out = self.bert.get_last_hidden_state(h_ids, h_seg)

        u = self.mean_pool(p_out, p_mask)
        v = self.mean_pool(h_out, h_mask)

        combined = torch.cat([u, v, torch.abs(u - v)], dim=1)
        return self.classifier(combined)

```

```
In [15]: raw_data = load_and_prepare_data()
```

```
In [16]: print(raw_data)
```

```

DatasetDict({
  train: Dataset({
    features: ['premise', 'hypothesis', 'label'],
    num_rows: 100000
  })
  validation: Dataset({
    features: ['premise', 'hypothesis', 'label'],
    num_rows: 5000
  })
})

```

```
In [17]: tokenized_data = raw_data.map(preprocess_function, batched=True)
```

```
In [18]: print(tokenized_data)
```

```

DatasetDict({
  train: Dataset({
    features: ['premise', 'hypothesis', 'label', 'premise_input_ids', 'p
remise_attention_mask', 'hypothesis_input_ids', 'hypothesis_attention_mask',
'labels'],
    num_rows: 100000
  })
  validation: Dataset({
    features: ['premise', 'hypothesis', 'label', 'premise_input_ids', 'p
remise_attention_mask', 'hypothesis_input_ids', 'hypothesis_attention_mask',
'labels'],
    num_rows: 5000
  })
})

```

Train loader and loading task1 Bert Model

```
In [19]: train_loader = DataLoader(tokenized_data['train'], batch_size=32, shuffle=True)
val_loader = DataLoader(tokenized_data['validation'], batch_size=32)
```

```
In [20]: BERT_MODEL = BERT(
    n_layers=12, # number of Encoder of Encoder Layer
    n_heads=12, # number of heads in Multi-Head Attention
    d_model=768, # Embedding Size
    d_ff=3072, # 4*d_model, FeedForward dimension
    d_k=64, # dimension of K(=Q), V
    n_segments=2,
    vocab_size=207419,
    max_len=128,
    device=device
)
```

```
In [21]: BERT_MODEL.load_state_dict(torch.load('best_bert_model.pth'),strict=False)
```

```
Out[21]: _IncompatibleKeys(missing_keys=['layers.0.enc_self_attn.layer_norm.weight',
'layers.0.enc_self_attn.layer_norm.bias', 'layers.1.enc_self_attn.layer_norm.weight',
'layers.1.enc_self_attn.layer_norm.bias', 'layers.2.enc_self_attn.layer_norm.weight',
'layers.2.enc_self_attn.layer_norm.bias', 'layers.3.enc_self_attn.layer_norm.weight',
'layers.3.enc_self_attn.layer_norm.bias', 'layers.4.enc_self_attn.layer_norm.weight',
'layers.4.enc_self_attn.layer_norm.bias', 'layers.5.enc_self_attn.layer_norm.weight',
'layers.5.enc_self_attn.layer_norm.bias', 'layers.6.enc_self_attn.layer_norm.weight',
'layers.6.enc_self_attn.layer_norm.bias', 'layers.7.enc_self_attn.layer_norm.weight',
'layers.7.enc_self_attn.layer_norm.bias', 'layers.8.enc_self_attn.layer_norm.weight',
'layers.8.enc_self_attn.layer_norm.bias', 'layers.9.enc_self_attn.layer_norm.weight',
'layers.9.enc_self_attn.layer_norm.bias', 'layers.10.enc_self_attn.layer_norm.weight',
'layers.10.enc_self_attn.layer_norm.bias', 'layers.11.enc_self_attn.layer_norm.weight',
'layers.11.enc_self_attn.layer_norm.bias'], unexpected_keys=[])
```

```
In [22]: model = SiameseBERT(BERT_MODEL, d_model=768).to(device)
optimizer = optim.Adam(model.parameters(), lr=2e-5)
criterion = nn.CrossEntropyLoss()
```

```
In [23]: import numpy as np
```

```
In [24]: import torch.cuda.amp as amp

num_epochs = 2
patience = 1
best_val_loss = float('inf')
patience_counter = 0

batch_size_per_step = 4
effective_batch_size = 32
accumulation_steps = effective_batch_size // batch_size_per_step

#clear any previous formatting to start fresh
tokenized_data['train'].reset_format()
tokenized_data['validation'].reset_format()
```

```

#redefine loaders
train_loader = DataLoader(tokenized_data['train'], batch_size=batch_size_per
val_loader = DataLoader(tokenized_data['validation'], batch_size=batch_size_

scaler = amp.GradScaler()
history = {'train_loss': [], 'val_loss': [], 'val_acc': []}

print(f"Starting Task 2 training on {device}...")

for epoch in range(num_epochs):
    model.train()
    total_train_loss = 0
    optimizer.zero_grad()

    train_loop = tqdm(train_loader, desc=f"Epoch {epoch+1}/{num_epochs} [Tra

    for i, batch in enumerate(train_loop):
        #handle list of lists or list of tensors manually
        # This fixes the TypeError and ensures we have the right types
        p_ids = torch.stack([torch.tensor(x, dtype=torch.long) for x in batch['p_ids']])
        p_mask = torch.stack([torch.tensor(x, dtype=torch.float) for x in batch['p_mask']])
        h_ids = torch.stack([torch.tensor(x, dtype=torch.long) for x in batch['h_ids']])
        h_mask = torch.stack([torch.tensor(x, dtype=torch.float) for x in batch['h_mask']])
        labels = torch.tensor(batch['labels'], dtype=torch.long).to(device)

        #if shape is (128, 4), transpose to (4, 128) -> This fixes the "128
        if p_ids.shape[0] == 128:
            p_ids, p_mask = p_ids.t(), p_mask.t()
            h_ids, h_mask = h_ids.t(), h_mask.t()

        with amp.autocast():
            logits = model(p_ids, p_mask, h_ids, h_mask)
            # Logits shape is now (4, 3), Labels shape is (4,)
            loss = criterion(logits, labels)
            loss = loss / accumulation_steps

        scaler.scale(loss).backward()

        if (i + 1) % accumulation_steps == 0 or (i + 1) == len(train_loader):
            scaler.step(optimizer)
            scaler.update()
            optimizer.zero_grad()

        total_train_loss += loss.item() * accumulation_steps
        train_loop.set_postfix(loss=loss.item() * accumulation_steps)

    del p_ids, p_mask, h_ids, h_mask, logits, labels

#val
model.eval()
val_loss, correct, total = 0, 0, 0
val_loop = tqdm(val_loader, desc=f"Epoch {epoch+1} [Val]")

with torch.no_grad():
    for batch in val_loop:
        p_ids = torch.stack([torch.tensor(x, dtype=torch.long) for x in batch['p_ids']])

```

```

        p_mask = torch.stack([torch.tensor(x, dtype=torch.float) for x in p_ids])
        h_ids = torch.stack([torch.tensor(x, dtype=torch.long) for x in h_ids])
        h_mask = torch.stack([torch.tensor(x, dtype=torch.float) for x in h_mask])
        labels = torch.tensor(batch['labels'], dtype=torch.long).to(device)

        if p_ids.shape[0] == 128:
            p_ids, p_mask, h_ids, h_mask = p_ids.t(), p_mask.t(), h_ids.t(), h_mask.t()

            with amp.autocast():
                logits = model(p_ids, p_mask, h_ids, h_mask)
                loss = criterion(logits, labels)

            val_loss += loss.item()
            preds = torch.argmax(logits, dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

            del p_ids, p_mask, h_ids, h_mask, logits, labels

    avg_val_loss = val_loss / len(val_loader)
    val_acc = correct / total
    print(f"\nEpoch {epoch+1}: Val Loss {avg_val_loss:.4f} | Val Acc {val_acc:.4f}")

    if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        torch.save(model.state_dict(), 'sbert_best_model.pth')
        patience_counter = 0
    else:
        patience_counter += 1

    if patience_counter >= patience:
        print("Early stopping triggered.")
        break

```

Starting Task 2 training on cuda:3...

```

Epoch 1/2 [Train]:  0%|          | 0/25000 [00:00<?, ?it/s]
Epoch 1 [Val]:    0%|          | 0/1250 [00:00<?, ?it/s]
Epoch 1: Val Loss 1.0855 | Val Acc 0.3558
Epoch 2/2 [Train]:  0%|          | 0/25000 [00:00<?, ?it/s]
Epoch 2 [Val]:    0%|          | 0/1250 [00:00<?, ?it/s]
Epoch 2: Val Loss 1.0821 | Val Acc 0.3812

```

```

In [26]: def predict_nli(premise, hypothesis, model, tokenizer, device):
        """
        Function to predict the relationship between two custom sentences
        without relying on external library tensor conversion.
        """
        model.eval()

        #tokenize without return_tensors (returns standard Python lists)
        p_data = tokenizer(premise,
                           padding='max_length',
                           max_length=128,
                           truncation=True)

        h_data = tokenizer(hypothesis,

```



```

padding='max_length',
max_length=128,
truncation=True)

#convert lists to tensors and add batch dimension [1, 128] -> wrap the
p_ids = torch.tensor([p_data['input_ids']], dtype=torch.long).to(device)
p_mask = torch.tensor([p_data['attention_mask']], dtype=torch.float).to(
h_ids = torch.tensor([h_data['input_ids']], dtype=torch.long).to(device)
h_mask = torch.tensor([h_data['attention_mask']], dtype=torch.float).to(

with torch.no_grad():
    #forward pass through SiameseBERT
    logits = model(p_ids, p_mask, h_ids, h_mask)
    prediction = torch.argmax(logits, dim=1).item()

# Map label ID to string
label_map = {0: "Entailment", 1: "Neutral", 2: "Contradiction"}

return label_map[prediction]

#Example
print("\nInference Examples")

examples = [
    ("A person is outdoors.", "A man is walking in the park."),
    ("The cat is sleeping on the sofa.", "The cat is running outside."),
    ("Two men are playing soccer.", "People are engaging in a sport.")
]

for p, h in examples:
    result = predict_nli(p, h, model, tokenizer, device)
    print(f"Premise: {p}")
    print(f"Hypothesis: {h}")
    print(f"Prediction: {result}\n" + "-"*30)

```

```

--- Custom Inference Examples ---
Premise: A person is outdoors.
Hypothesis: A man is walking in the park.
Prediction: Neutral
-----
Premise: The cat is sleeping on the sofa.
Hypothesis: The cat is running outside.
Prediction: Entailment
-----
Premise: Two men are playing soccer.
Hypothesis: People are engaging in a sport.
Prediction: Contradiction
-----

```

```

In [33]: from sklearn.metrics import classification_report
import torch

def run_detailed_evaluation(model, loader, device):
    model.eval()
    all_preds = []
    all_labels = []

```

```

print("Gathering predictions for detailed report...")
with torch.no_grad():
    for batch in tqdm(loader, desc="Evaluating"):
        #pure PyTorch conversion to avoid environment errors
        p_ids = torch.stack([torch.tensor(x, dtype=torch.long) for x in
        p_mask = torch.stack([torch.tensor(x, dtype=torch.float) for x in
        h_ids = torch.stack([torch.tensor(x, dtype=torch.long) for x in
        h_mask = torch.stack([torch.tensor(x, dtype=torch.float) for x in
        labels = torch.tensor(batch['labels'], dtype=torch.long).to(device)

        #orientation Fix (128 vs Batch)
        if p_ids.shape[0] == 128:
            p_ids, p_mask, h_ids, h_mask = p_ids.t(), p_mask.t(), h_ids.t(), h_mask.t()

        logits = model(p_ids, p_mask, h_ids, h_mask)
        preds = torch.argmax(logits, dim=1)

        #move to CPU and store as lists
        all_preds.extend(preds.cpu().tolist())
        all_labels.extend(labels.cpu().tolist())

    #map numbers back to names
    target_names = ["Entailment", "Neutral", "Contradiction"]

    #generate the report
    report = classification_report(all_labels, all_preds, target_names=target_names)
    return report

#run it on your validation
full_report = run_detailed_evaluation(model, val_loader, device)

print("\nTask 3: Classification Report")
print(full_report)

```

```

Gathering predictions for detailed report...
Evaluating:  0%|          | 0/1250 [00:00<?, ?it/s]
--- Task 3: Classification Report ---

```

	precision	recall	f1-score	support
Entailment	0.46	0.20	0.28	1713
Neutral	0.38	0.52	0.44	1626
Contradiction	0.35	0.44	0.39	1661
accuracy			0.38	5000
macro avg	0.40	0.38	0.37	5000
weighted avg	0.40	0.38	0.37	5000

In []: